

Department of Computer Science and Engineering

1. Subject Code: **MOS-601** Course Title: **System Platforms**
2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks
3. **Prerequisites**
 - SWE-401, SWE-502
4. **Course Structure**
5 Units $\times$ 30 Hours (lectures, source-code reading, laboratory, PoW)

1. Course Description

MOS-601 closes the systems track by moving from a single Linux host to an operated fleet. Students provision reproducible infrastructure as code, extend Kubernetes with their own controllers, instrument systems for SLO-driven reliability, engineer the distributed networking, storage and consensus layers underneath, and then run the resulting platform through chaos exercises, incident command, supply-chain hardening and cost governance. The course is assessed on operational evidence: post-mortems, SLO reports, capacity models and signed release artefacts.

2. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Provision reproducible multi-environment infrastructure using Infrastructure as Code and GitOps workflows.
- CO2.** Operate and extend Kubernetes at production scale, including custom controllers and admission policy.
- CO3.** Implement full-stack observability and manage reliability through SLIs, SLOs and error budgets.
- CO4.** Design distributed networking, storage and consensus layers with explicit failure and consistency trade-offs.
- CO5.** Practise incident response, chaos engineering, supply-chain security and cost governance in production.

3. Detailed Syllabus

Unit	Contents	Hrs
1	Infrastructure as Code, GitOps & Platform Foundations <i>Module A: Declarative Infrastructure</i> Terraform/OpenTofu core model: providers, state, locking, modules, and drift detection. Immutable machine images. Cloud primitives: virtual networks, identity and access management, object storage, managed databases, and load balancers. <i>Module B: GitOps Delivery</i> Pull-based reconciliation with Argo CD or Flux. Environment promotion from development to staging to production, configuration overlays, and rollback semantics. Policy checks in the pipeline. <i>Module C: Secrets and the Internal Developer Platform</i> Secret management with Vault, SOPS and external-secrets operators; rotation and dynamic credentials. Golden paths, service scaffolding, self-service catalogues, and platform-as-product thinking. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Provision a complete multi-environment footprint entirely in code, with automated drift detection running in CI and a documented state-recovery procedure. • PoW: Build a GitOps promotion pipeline that moves an artefact from development to production with approval gates and verifiable rollback. • Migrate hard-coded credentials to dynamic, short-lived secrets and prove no plaintext remains in the repository history.	30
2	Kubernetes Internals, Extensibility & Workload Engineering <i>Module A: Control Plane Internals</i> API server request flow, etcd storage and watch semantics, the scheduler and its predicates/priorities, controller-manager reconciliation loops. CRI, CNI and CSI interfaces. <i>Module B: Workload Placement and Scaling</i> Requests and limits, quality-of-service classes, affinity and anti-affinity, topology spread, taints and tolerations, priority and preemption. Horizontal, vertical, event-driven (KEDA) and cluster autoscaling. StatefulSets and storage lifecycle. <i>Module C: Extending the Cluster</i> Custom Resource Definitions and the operator pattern with controller-runtime. Admission control: validating and mutating webhooks, OPA Gatekeeper and Kyverno policy. Multi-tenancy, namespaces, quotas and network policy. Packaging with Helm and Kustomize.	30

Unit	Contents (continued)	Hrs
	Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Implement a custom Kubernetes operator with a CRD and reconcile loop that manages the full lifecycle of a stateful workload, including backup and failover. • PoW: Enforce organisational policy through an admission webhook and demonstrate rejection of non-compliant manifests in CI. • Tune scheduling constraints to improve bin-packing and evidence node-count reduction.	
3	Observability, SLOs & Performance Engineering at Scale <i>Module A: Signals</i> Metrics with Prometheus and PromQL, cardinality control and recording rules. Structured logging, sampling and retention with Loki or the ELK stack. Distributed tracing with OpenTelemetry: context propagation, span design and tail-based sampling. Continuous profiling with eBPF-based tooling. <i>Module B: Reliability Targets</i> Choosing service level indicators, defining service level objectives, error budgets and burn-rate alerting. Alert design that avoids fatigue, on-call rotation hygiene, and runbook authoring. <i>Module C: Performance and Capacity</i> USE and RED methods. Load and soak testing with k6, benchmarking methodology, saturation analysis, queueing intuition (Little's Law), and capacity models with headroom planning. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Instrument a multi-service system end-to-end with OpenTelemetry, define SLOs with multi-window burn-rate alerts, and demonstrate detection of an injected regression. • PoW: Produce a capacity model from load-test data with a stated saturation point and scaling recommendation. • Reduce metric cardinality by an order of magnitude without losing alerting fidelity.	30

Unit	Contents (continued)	Hrs
4	Distributed Networking, Storage & Consensus <i>Module A: Service Networking</i> Envoy proxy fundamentals; service mesh with Istio or Linkerd: mTLS, retries, timeouts, circuit breaking, traffic shifting. Gateway API and ingress design. DNS, load-balancing algorithms, anycast, and CDN/edge placement. <i>Module B: Consensus and Replication</i> The Raft protocol: leader election, log replication, membership change. Quorum systems, replication topologies, partitioning and rebalancing. CAP and PACELC in operational terms. Distributed transactions, sagas and idempotent compensation. <i>Module C: Storage and Continuity</i> Block, file and object storage characteristics. Distributed storage systems (Ceph, MinIO). Backup strategy, restore verification, RPO/RTO targets, and multi-region and disaster-recovery topologies. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Deploy a service mesh with mutual TLS and execute a canary traffic shift with automated rollback on SLO violation. • PoW: Build a Raft-backed configuration store and validate correct leader failover and no split-brain under an injected network partition. • Execute a full backup and restore drill and report measured RPO and RTO against target.	30
5	Reliability, Security & FinOps in Production <i>Module A: Incident Management</i> Incident command roles, severity classification, communication protocol, and blameless postmortem practice. Root cause analysis methods, contributing-factor analysis, and action-item tracking. <i>Module B: Resilience and Delivery Risk</i> Chaos engineering: hypothesis-driven fault injection with Chaos Mesh or LitmusChaos, GameDay design. Progressive delivery: canary, blue-green, feature flags, and automated rollback triggers. <i>Module C: Supply Chain, Runtime Security & Cost</i> SBOM generation, artefact signing with Sigstore/cosign, provenance and SLSA levels, dependency and image scanning. Runtime security with Falco and eBPF, seccomp/AppArmor profiles at fleet scale, zero-trust networking, and compliance basics. Cost engineering: rightsizing, spot capacity, bin-packing, commitment planning, and showback/chargeback. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Run a GameDay chaos exercise against the platform as incident commander and publish a blameless postmortem with tracked action items. • PoW: Deliver a signed, SBOM-attested release pipeline with policy-enforced admission of only verified artefacts. • PoW: Produce a cost-optimisation report demonstrating a measured reduction in spend at unchanged SLO attainment.	30

Unit	Contents (continued)	Hrs
Total Instructional Effort		150

4. Recommended Resources

- Beyer et al., *Site Reliability Engineering* and *The Site Reliability Workbook*.
- Burns, Beda & Hightower, *Kubernetes: Up and Running; Programming Kubernetes*.
- Ongaro & Ousterhout, *In Search of an Understandable Consensus Algorithm* (Raft).
- Gregg, *Systems Performance*, 2nd ed.; official OpenTelemetry documentation.
- Terraform, Argo CD, Istio, Sigstore and SLSA official documentation.